

10.3.3 工程版源码走读：第三节课代码全景串讲

这份文档怎么用

第三节课（Nav2 导航）的配套文档一共三份，各管一段：

1. 《10.3.3_Nav2导航系统与路径规划》——**原理**：AMCL、代价地图、规划器是怎么回事；
2. 《10.3.3_动手实现教程_从零写出导航集成层》——**动手**：你自己从零写出简化版的激活器、安全门、任务执行器；
3. 本文——**走读**：拿着仓库里的**工程版源码**逐个文件串讲，重点讲"工程版比你写的简化版多了什么、为什么多"。

推荐先完成动手教程再读本文：你自己写过一遍，才能体会工程版每一处"多余"代码背后都是一个真实踩过的坑。没写过也可以读，但遇到"对照你的 v2"这类字眼时请回动手教程补课。

走读时请开着编辑器对照真实文件，本文引用的行号以当前仓库代码为准。

第 0 章 全景：这节课到底有哪些代码

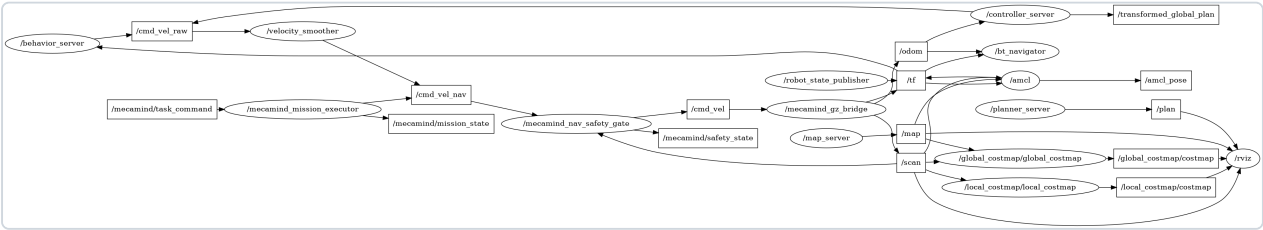
第三节课的全部代码就是下面 9 个文件，总共约 2600 行（其中一半是注释——工程版源码本身就写成了教材）：

文件	行数	角色	ROS 2 机制	动手教程对应章
<code>src/mecamind_bringup/launch/navigation.launch.py</code>	306	总装图纸：一条命令拉起全系统	launch + remapping	第 7 章
<code>src/mecamind_tools/mecamind_tools/nav_utils.py</code>	110	工具箱：被 3 个节点共用的纯函数	无	第 3 章
<code>src/mecamind_tools/mecamind_tools/lifecycle_activator.py</code>	251	点火钥匙：激活 9 个 Nav2 节点	service 客户端	第 4 章
<code>src/mecamind_tools/mecamind_tools/safety_layer.py</code>	420	安全门：速度链最后一关	topic 订阅 + 发布	第 5 章
<code>src/mecamind_tools/mecamind_tools/mission_executor.py</code>	583	任务翻译官：JSON 指令 → Nav2 goal	action 客户端	第 6 章
<code>src/mecamind_tools/mecamind_tools/waypoint_patrol.py</code>	139	巡航示例：同一件事的同步写法	BasicNavigator	第 6.1 节
<code>src/mecamind_tools/config/mecamind_nav2_params.yaml</code>	372	Nav2 全栈参数	—	(未覆盖)
<code>src/mecamind_tools/mecamind_tools/nav_acceptance.py</code>	324	自动考官：CP3 验收节点	topic 黑盒测试	(未覆盖)
<code>scripts/test_cp3.sh</code>	125	验收脚本：无人值守跑完整验收	shell	(未覆盖)

外加两个小配置文件：`mecamind_named_goals.yaml`（命名目标点表）和 `mecamind_patrol_waypoints.yaml`（巡航路线）。

0.1 先看一张"活的架构图"

架构图人人会画，但下面这张不是画的——它是系统真实跑起来后 `rqt_graph` 生成的节点/话题连接图。读懂这张图，等于读懂了整个第三节课：



rqt_graph：导航系统真实运行时的节点与话题连接图

照着图从左往右走三条线（与动手教程第 0 章的"三条线、一把钥匙"完全对应）：

- **任务线**（图左）：`/mecamind/task_command` → `mecamind_mission_executor` → （action，图上不显示）→ `bt_navigator`；执行器同时向 `/mecamind/mission_state` 汇报进度。
- **控制线**（图中）：`controller_server` 和 `behavior_server` 发 `/cmd_vel_raw` → `velocity_smoother` 发 `/cmd_vel_nav` → `mecamind_nav_safety_gate` 发 `/cmd_vel` → 仿真桥（Gazebo 后端是 `mecamind_gz_bridge`）。注意看：**能直接摸到 `/cmd_vel` 的只有安全门一个节点**，这就是这条链的设计目的。
- **数据线**（图右）：`/scan`、`/odom`、`/tf` 从仿真器流向 `amcl`、两张代价地图和 RViz；`map_server` 发 `/map`。

图上还有个细节值得指给学员看：`/scan` 同时喂给了 `AMCL`、`global_costmap`、`local_costmap` 和安全门四个消费者——topic 的"一发多收"广播特性在真实系统里就长这样。

0.2 建议的走读顺序

本文按依赖关系从外到内安排章节：**launch（骨架）** → **nav_utils（地基）** → **三个节点（肌肉）** → **参数（调校）** → **验收（考卷）**。每章开头有"结构地图"，中间是重点段落精讲，结尾有一个"走读自查"问题。

第 1 章 navigation.launch.py——总装图纸走读

打开 `src/mecamind_bringup/launch/navigation.launch.py`。动手教程第 7 章已经讲过它的三个灵魂（remapping 拼速度链、TimerAction 延迟点火、双后端开关），本章把整个文件从头到尾串一遍，补上没讲的细节。

1.1 文件结构地图

```
L50 NAV2_LIFECYCLE_NODES    ← 模块级常量：9 个节点的激活顺序表
L63 generate_launch_description()
    L68~78  声明 8 个 LaunchConfiguration + 两个后端条件
    L81~96  双后端：gazebo_backend / lite_backend（二选一 Include）
    L99     nav2_common      ← 所有 Nav2 节点共用的参数写法
    L103~185 9 个 Nav2 节点（map_server ... velocity_smoother）
    L189~222 2 个自研节点（safety_gate、mission_executor）
    L229~247 lifecycle_activator（包在 TimerAction 里）
    L249~257 RViz（with_rviz 条件启动）
    L259~305 LaunchDescription：声明参数默认值 + 装配所有节点
```

1.2 NAV2_LIFECYCLE_NODES：顺序就是依赖关系

```
NAV2_LIFECYCLE_NODES = [
    "map_server",          # 先有地图
    "amcl",                # 才能在地图上定位
    "controller_server",   # 而后规划/控制/行为
    "smoother_server",
    "planner_server",
    "behavior_server",
    "bt_navigator",        # 总指挥依赖以上全部
    "waypoint_follower",
    "velocity_smoother",
]
```

这个列表会原样作为参数传给 `lifecycle_activator`（第 238 行 `{"node_names": NAV2_LIFECYCLE_NODES}`）。把它提成模块级常量而不是写死在激活器里，是为了让**"激活哪些节点、按什么顺序"**这个系统级决策留在总装图纸上——激活器只是执行者，它的默认参数表里甚至没有 `map_server`（对照 `lifecycle_activator.py` 第 52~64 行），是 `launch` 把清单喂给它的。

1.3 nav2_common：参数的"叠加"写法

```
nav2_common = [params_file, {"use_sim_time": True}]
```

`parameters` 列表是**从左到右依次加载、后面覆盖前面**的。这一行的含义是：先加载整个 `mecamind_nav2_params.yaml`，再无条件把 `use_sim_time` 钉成 `True`。即使有人改参数文件时误删了某个节点的 `use_sim_time`，仿真时间也不会被破坏——这是防御性的双保险。

1.4 速度链是"改名字"改出来的（三处 remapping 汇总）

节点	remapping	效果
<code>controller_server</code> (L128)	<code>cmd_vel → /cmd_vel_raw</code>	DWB 的输出进链
<code>behavior_server</code> (L154)	<code>cmd_vel → /cmd_vel_raw</code>	恢复行为（倒车/旋转）也发速度，漏了它就有旁路
<code>velocity_smoother</code> (L181~184)	<code>cmd_vel → /cmd_vel_raw</code> ， <code>cmd_vel_smoothed → /cmd_vel_nav</code>	输入接链、输出改名交给安全门

安全门自己不用 `remap`——它的输入/输出 `topic` 本来就是参数（L197~198 直接配 `/cmd_vel_nav → /cmd_vel`）。两种接线方式的区别值得体会：**别人的节点用 `remapping` 改名，自己写的节点直接做成参数。**

最容易被忽略的是 `behavior_server` 那一处。课堂上可以做个思想实验：如果漏掉它，机器人平时一切正常，直到某次导航卡住触发"倒车脱困"——倒车速度绕过平滑器和安全门直达底盘。**旁路 bug 的可怕之处是平时测不出来。**

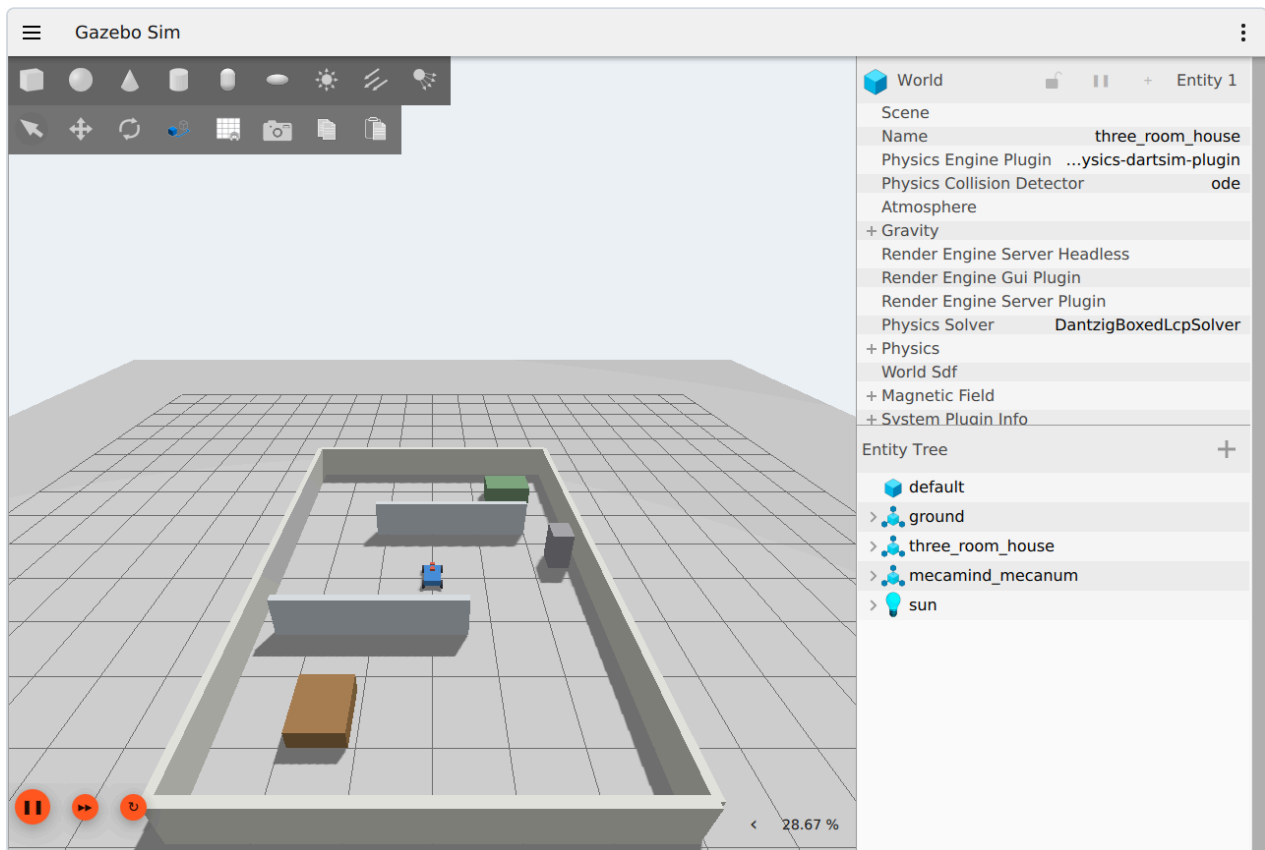
1.5 为什么激活器要包在 TimerAction(period=8.0) 里

```
lifecycle_activator = TimerAction(  
    period=8.0,  
    actions=[Node(..., parameters=[..., {"service_timeout_sec": 60.0}, ...]),  
    )
```

launch 启动所有进程几乎是同时的，但 Gazebo 冷启动到 /clock 有数据要好几秒； Nav2 各节点创建 change_state service 也要时间。8 秒延迟 + 60 秒 service 超时是双重容错：延迟让大部分情况下第一次尝试就成功，长超时兜住 Gazebo 特别慢的那次。注意激活器还把初始位姿参数覆盖成了 (0, 0, 0) ——因为两个后端的机器人出生点统一在原点，而第二课的地图就是从原点建起的，map 坐标系与世界坐标系近似重合（L87~94 lite 后端的注释讲的就是这件事）。

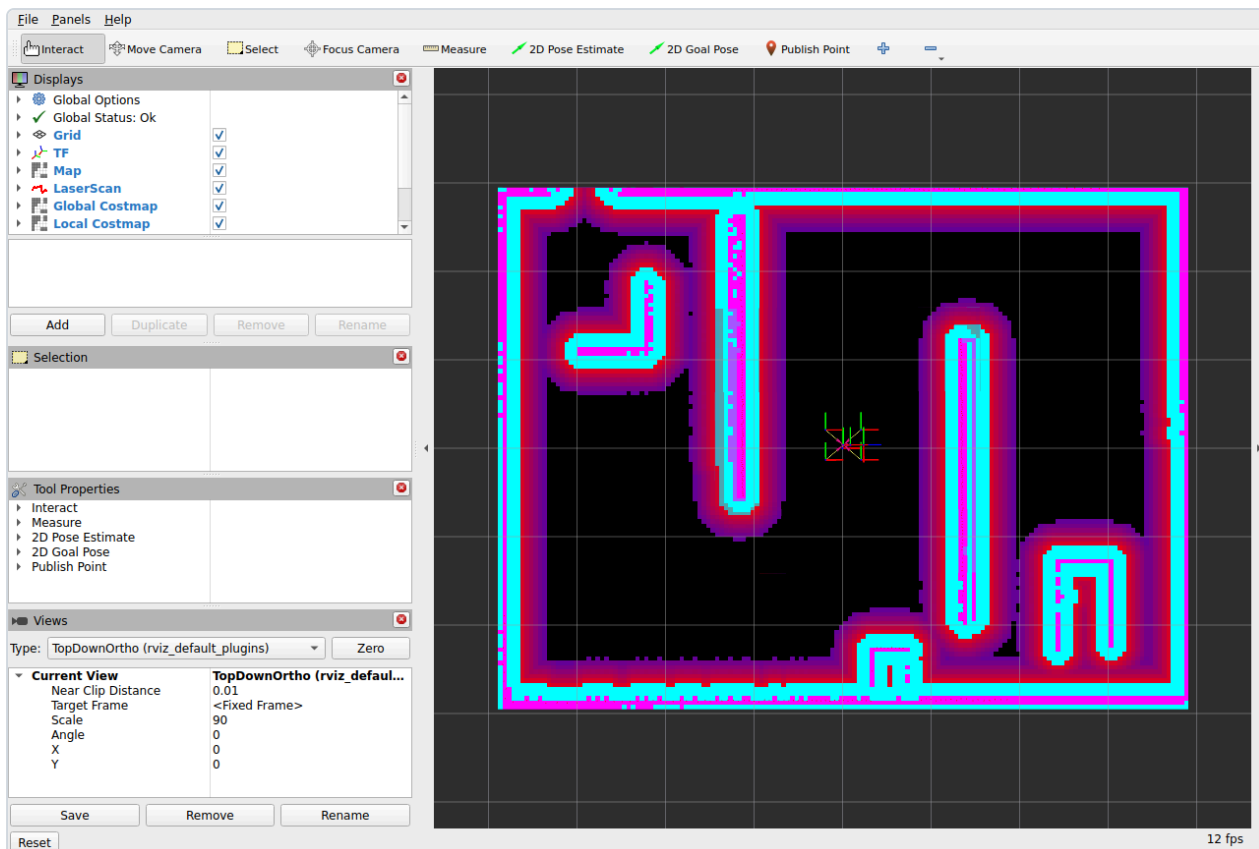
1.6 启动后你应该看到什么

Gazebo 后端启动完成后的两个窗口（真实截图）：



Gazebo：三室户世界与麦轮机器人

RViz 里地图、全局/局部代价地图正常渲染，机器人 TF 出现在原点，说明 map_server 和 AMCL 已激活：



RViz：导航系统启动完成，青色为障碍、紫红色为膨胀带

这张 RViz 截图本身就值得走读：青色是障碍物本体（激光打到的墙和家具），外面包的紫红色渐变是 `inflation_layer` 渲染的**膨胀带**（第 7 章会讲它的参数），规划器会自动避开代价高的区域，所以路径永远不会贴着墙走。

走读自查： `waypoint_follower` 和 `smoother_server` 没有任何 `remapping`，为什么它们不需要进速度链？（提示：它们的输出是什么类型的数据？谁消费？）

第 2 章 `nav_utils.py`——被三个节点共用的地基

110 行的小文件，但 `lifecycle_activator`、`mission_executor`、`waypoint_patrol` 三个节点都 `from .nav_utils import ...`。动手教程第 3 章你已经写过 `quaternion_from_yaw`，这里补讲另外三个函数。

2.1 `load_yaml`：为什么读个 YAML 也要封装

```
def load_yaml(path: str) -> Dict[str, Any]:
    expanded = os.path.expanduser(path)          # ① "~/maps/x.yaml" 展开家目录
    with open(expanded, "r", encoding="utf-8") as stream:
        data = yaml.safe_load(stream) or {}        # ② 空文件当空字典，safe_load 防注入
    if not isinstance(data, dict):
        raise ValueError(f"YAML root must be a mapping: {path}") # ③ 顶层必须是映射
    return data
```

三件小事（~ 展开、空文件兜底、顶层类型检查）单独看都微不足道，但如果每个节点各写一份，行为不一致的那天就是排查半小时的那天。“把小事收拢到一处”是工具模块存在的全部理由。

2.2 pose_stamped_from_dict: Stamped 的分量

```
pose = PoseStamped()
pose.header.stamp = stamp                # 时间戳由调用方传入!
pose.header.frame_id = str(item.get("frame_id", "map"))
pose.pose.position.x = float(item["x"])  # x、y 必填，缺了直接 KeyError
pose.pose.orientation = quaternion_from_yaw(float(item.get("yaw", 0.0)))
```

两个设计决定值得停下来想：

1. **stamp 为什么不在函数内部取"当前时间"?** 因为"当前时间"取决于时钟来源——仿真里必须用节点的 ROS 时钟（`node.get_clock().now()`，跟随 `/clock`），而纯函数没有节点、摸不到 ROS 时钟。把时间戳做成参数，函数保持纯粹，时钟责任归调用方。`mission_executor` 里有一处对应细节：目标在队列里可能排了一阵子，**发送时才现生成时间戳**（`_send_pending` 第 447 行），用排队前的旧时间戳会导致 TF 查询失败。
2. **x、y 用 `item["x"]` 硬取，yaw 用 `.get(..., 0.0)` 软取**——必填项配置错了要"尽早崩、崩得响"，可选项才给默认值。错误处理的松紧是刻意设计的，不是随手写的。

`initial_pose_from_dict` 的实现与它逐行相同，单独存在只为了让调用处读起来语义清晰：一个用于"要去哪"，一个用于"我在哪"。用重复换可读性，在 20 行的小函数上是划算的交易。

走读自查：如果把 `quaternion_from_yaw` 里的 `yaw * 0.5` 写成 `yaw`，机器人到达目标后朝向会差多少？

第 3 章 lifecycle_activator.py——比你的 my_activator 多了什么

动手教程第 4 章你写的 `my_activator` 已经覆盖了主干：`get_state` / `change_state` 两个 service 客户端 + "先查状态再补迁移"。工程版多出来的每一块都值得单独讲。

3.1 差异总表

#	工程版多出的东西	防的是什么
1	<code>/initialpose</code> 用 <code>TRANSIENT_LOCAL</code> QoS 发布	AMCL 订阅晚了错过初始位姿
2	初始位姿重复发 5 次、带协方差	单发被丢 / 粒子云不收敛
3	<code>post_initial_pose_delay_sec</code> 再等 2 秒	下游节点激活时 <code>map→odom</code> 还不存在
4	<code>ok = node_ok and ok</code> 不提前 <code>return</code>	修一个 bug 跑一次, 修 9 个跑 9 次
5	幂等处理 (已 <code>active</code> 直接跳过)	重复运行激活器把系统搞坏
6	<code>SystemExit(0/1)</code> + <code>rclpy.ok()</code> 判断	脚本无法判断激活是否成功 / <code>Ctrl+C</code> 时二次崩溃

3.2 精讲一: TRANSIENT_LOCAL——"给迟到者留一份"

```
qos = QoSProfile(depth=1)
qos.durability = DurabilityPolicy.TRANSIENT_LOCAL
self.initial_pose_pub = self.create_publisher(PoseWithCovarianceStamped, "/initialpose", qos)
```

默认的 `VOLATILE` 策略是"发出即忘": 发布时刻没在场的订阅者永远收不到。 `TRANSIENT_LOCAL` 则让发布者为后来的订阅者保留最后一条消息。 激活器发初始位姿的时机紧挨着 `AMCL` 刚激活, `AMCL` 内部订阅器是否已就绪是个竞态—— 这个 QoS 让竞态无害化。 `/map` 话题也是同款设计 (所以 `RViz` 晚开也能看到地图), 参数文件里 `map_subscribe_transient_local: true` 就是订阅侧的配合。

3.3 精讲二: 协方差的三个数字

```
msg.pose.covariance[0] = 0.25    # x 方差 → 标准差 0.5 米
msg.pose.covariance[7] = 0.25    # y 方差 (6x6 矩阵第 2 行第 2 列, 7 = 1*6+1)
msg.pose.covariance[35] = 0.0685 # yaw 方差 ≈ (15°)²
```

给 `AMCL` 的初始位姿不是"你就在这", 而是"你大概在这, 误差半米、朝向误差 15° 以内"。 粒子滤波需要这点不确定度才会把粒子撒开、靠激光匹配收敛到真实位置; 协方差全填 0 等于让 `AMCL` 盲信给定坐标, 初始位姿稍有偏差它反而纠不回来。

3.4 精讲三: 不短路的错误处理

```
for node_name in node_names:
    node_ok = self.configure_and_activate(node_name)
    ok = node_ok and ok          # 失败不 break, 继续激活剩下的
```

写成 `if not node_ok: return False` 也"对", 但排障体验天差地别: 9 个节点里 3 个配置有问题时, 短路版要"修一个、跑一次"来回 3 遍; 这个写法一次运行就把所有故障点全部打印出来。

另外注意工程版特意把 `configure_and_activate` 的调用单独提一行求值, 再参与 `and`。 如果有人"顺手简化"成 `ok = ok and self.configure_and_activate(node_name)`, Python 的 `and` 短路特性会让 `ok` 一旦变 `False`

后跳过所有后续节点的激活调用——一行看似等价的重构，把"继续激活余下节点"的设计整个废掉了。这是布尔短路最经典的埋雷姿势。

走读自查： `configure_and_activate` 里 `configure` 成功后为什么要重新查询一次状态再决定是否 `activate`，而不是直接 `activate`？（提示：`configure` 的 `service` 返回 `success` 只代表"迁移被接受"，看 L177~178 注释。）

第 4 章 `safety_layer.py`——比你的 `my_safety_gate` 多了什么

动手教程第 5 章的 `my_safety_gate` 只看正前方一个扇区、只有"停/不停"一级判断。工程版 420 行里一半是纯函数（可以脱离 ROS 做单元测试），一半是接线，结构本身就是教学范本。

4.1 结构地图

纯函数区（不 import rclpy，可直接单测）

L43

SectorDistances

四扇区最近距离（数据类）

L57

SafetyDecision

一次评估的完整结论（数据类，可转 JSON）

L85

_angle_diff

角度差归一化（atan2 技巧）

L94

_sector_min

某扇区内最近障碍距离

L125

analyze_scan

★ 核心：一帧激光 → 一份决策

L202

gate_twist

★ 核心：决策作用到 Twist 各分量

L232

command_timed_out

指令超时判断

节点区（只做接线）

L247

SafetyGateNode

订阅 scan+cmd、发布 cmd+state、20Hz 看门狗

4.2 差异总表

#	工程版多出的东西	你的简化版
1	前/后/左/右四个扇区，左右管麦轮横移	只看正前方
2	stop/warn 两级阈值，warn 区线性衰减（保底 0.25）	只有一刀切的停止线
3	倒车不限、旋转不限的分量原则	全部一起清零
4	两个看门狗：激光断流 + 指令断流都急停	没有
5	决策以 JSON 发到 <code>/mecamind/safety_state</code>	没有（验收脚本靠它取证）
6	阈值参数每次现读， <code>ros2 param set</code> 立即生效	启动时读一次

4.3 精讲一：_angle_diff——一行 atan2 解决绕圈问题

```
def _angle_diff(a: float, b: float) -> float:
    return math.atan2(math.sin(a - b), math.cos(a - b))
```

判断"某束激光是否落在后方扇区"时，直接比较角度会撞上 359° 与 1° 差 358° 的绕圈问题—— 后方扇区中心是 180° ，恰好横跨 $\pm\pi$ 的分界。先取 $\sin/\cos$ 再 atan2 还原， 角度差被自动折回 $(-\pi, \pi]$ ，跨界扇区不再需要特判。这个技巧在机器人代码里出镜率极高，值得背下来。

4.4 精讲二：warn 区的线性衰减为什么要保底

```
elif sectors.front < front_warn_dist:
    span = max(0.001, front_warn_dist - front_stop_dist)
    scale_x = max(0.25, min(1.0, (sectors.front - front_stop_dist) / span))
```

从 warn 线 (0.45m) 到 stop 线 (0.25m)，前进速度按距离线性从 1.0 衰减——但保底 0.25。没有保底会发生什么？机器人在临界距离上速度趋近 0、几乎不动，控制器的进度检查 (12 秒没挪动 0.35m) 判定"卡住"、触发恢复行为，倒车再来，无限循环。 **0.25 这个数字是被"机器人在门口反复鬼畜"的现场调出来的。** 另外注意 `max(0.001, span)`：两条线被配得相等时防除零——参数是用户给的，不可信。

4.5 精讲三：gate_twist 的分量原则

```
if command.linear.x > 0.0:
    gated.linear.x = command.linear.x * decision.scale_x # 只限"前进"
if command.linear.y > 0.0 and not decision.allow_left:
    gated.linear.y = 0.0 # 向左横移看左侧
elif command.linear.y < 0.0 and not decision.allow_right:
    gated.linear.y = 0.0 # 向右横移看右侧
# angular.z 完全不动——旋转是脱困必需的动作
```

三条原则各有道理：倒车不限，是为了被堵住时还能退出来； 旋转不限，是因为原地转不产生平移位移、且恢复行为靠它脱困； 横移分侧管理，是麦轮底盘特有的需求（差速车根本没有 y 分量）。 **安全门的哲学是"修剪"而不是"否决"**——把危险的分量剪掉，无害的分量放行， 机器人被拦下时仍保有自救能力。

4.6 精讲四：两个看门狗，两种失明

```
# 看门狗 1: 激光断流 (_decision 内, L319)
if self._last_scan is None or time.monotonic() - self._last_scan_wall_time > timeout:
    return SafetyDecision(level="stop", reason="scan_stale", ...)

# 看门狗 2: 指令断流 (20Hz 定时器 _command_watchdog, L362)
# 上游崩溃后底盘会"锁"在最后一条速度上狂奔，补发一帧零 Twist 停住
```

- **传感器失明**（雷达掉线）：拒绝放行任何运动——看不见就不动；
- **大脑失联**（Nav2/遥控崩溃）：主动发零速度——最后一条非零指令不能让它跑到天荒地老。

两处计时都用 `time.monotonic()`（单调墙钟）而不是 ROS 时间，原因在 L272 注释： 仿真暂停时 `sim time` 停走，但"激光断流"的检测不应该跟着停。 **判断"数据流是否还活着"永远用墙钟，判断"物理量何时发生"才用 ROS 时钟**——这个区分后面课程还会反复用到。

还有个细节：看门狗触发后先把 `_command_active` 置回 False 再发急停 (L379)， 保证零速度只补发一次，而不是 20Hz 无限刷屏。

走读自查： `_cmd_cb` 里判断"机器人在动"用的是**修剪后**的 `gated` 值而不是原始指令（L352），换成原始指令会有什么后果？（提示：前方有墙、上游持续发前进指令、然后上游崩溃——看门狗还会武装吗？需要吗？）

第 5 章 `mission_executor.py`——比你的 `my_executor_v2` 多了什么

这是全课最长的文件（583 行），也是动手教程 6.3 节埋的伏笔："三个被时间打过的补丁"。本章把三个补丁的代码真身找出来，再补上 6.3 没展开的业务层。

5.1 结构地图

纯函数/数据类区

L45 `MissionState` 状态快照（frozen dataclass → JSON）
L70 `normalize_goal_key` 目标名归一化（"Living Room" == "living_room"）
L79 `load_named_goals` 读命名目标表
L104 `resolve_named_goal` 查表 + 中文别名（"厨房" → kitchen）
L133 `load_patrol_waypoints`
L146 `next_mission_mode` intent → 模式（5 行的状态机核心）

节点区

L167 `MissionExecutorNode`
L282 `_task_cb` 指令入口（大脑）
L326 `_stop_all` / `_enter_follow` / `_enter_navigate` / `_enter_patrol`
L386 `_cancel_nav` ★ 补丁 2：序列号防串扰
L409 `_tick` ★ 补丁 1：两阶段发送的驱动器
L436 `_send_pending` → L467 `_goal_response` → L519 `_goal_finished` 三环回调链
L501 `_poll_active_goal` ★ 补丁 3：超时兜底

5.2 补丁一：两阶段发送——指令永不丢失

你的 `v2` 里 `wait_for_server` 超时后指令就丢了。工程版把"收指令"和"发 goal"拆开：

```
# _enter_navigate: 收到指令只是"排队"
self._pending_send = {"kind": "navigate", "pose": goal, "name": ...}

# _tick (0.5 秒定时器): 确认服务端就绪才真正发送
if not self.navigate_client.server_is_ready():
    self._detail = "waiting_for_nav2"    # 下个周期再试
    self._publish_state()
    return
self._send_pending()
```

好处一句话：用户在 Nav2 启动完成前 30 秒发的指令，也会在 Nav2 就绪那一刻自动发出。课堂演示时这直接决定体验——你不用掐着表等系统就绪才敢喊"去厨房"。

`_tick` 同时兼三份工：心跳重发状态（晚启动的观察者随时能拿到当前状态）、驱动排队目标、巡检超时。一个 0.5 秒的定时器盘活整个节点，这种"轮询驱动 + 异步回调"混合风格是 `rclpy` 单线程模型下的实用套路。

5.3 补丁二： `_goal_serial`——给回调"盖章验身"

异步系统的经典 bug：旧 goal 刚被取消，它的 `result` 回调还在路上；新 goal 已经 active；旧回调到达，把状态错写成旧目标的结果。工程版的解法只有三步：

```
# ① 每次发送/取消，序列号 +1 (_cancel_nav 第一行、_send_pending L453)
self._goal_serial += 1
serial = self._goal_serial

# ② 注册回调时用 lambda 默认参数把当前序列号"焊"进闭包
send_future.add_done_callback(
    lambda future, name=..., goal_serial=serial: self._goal_response(future, name, ..., goal_serial)
)

# ③ 回调进门先验章 (_goal_response / _goal_finished 第一行)
if serial != self._goal_serial:
    return # 过期回调，直接作废
```

第 ② 步的 `goal_serial=serial` 是 Python 闭包的经典技巧：`lambda` 的默认参数在**定义时刻**求值，把当时的值捕获进去；直接写 `lambda future: self._goal_response(future, ..., self._goal_serial)` 则会在**回调执行时刻**才去读变量，那时序列号早变了——章就白盖了。

动手教程 6.3 的那句话在这里落地：异步系统里，"迟到的回调"是万恶之源，序列号是最便宜的解药。

5.4 补丁三： `_poll_active_goal`——上层兜底

```
if now - self._goal_sent_time < self._goal_timeout:
    return
self.get_logger().warn("Navigate goal timed out; canceling")
self._cancel_nav()
self.cmd_pub.publish(Twist()) # 不等 Nav2 收尾，先急停
self._set_state("idle", "goal_timeout")
```

机器人被困住时，Nav2 内部会"规划失败 → 恢复行为 → 再规划"无限循环，迟迟不返回结果。

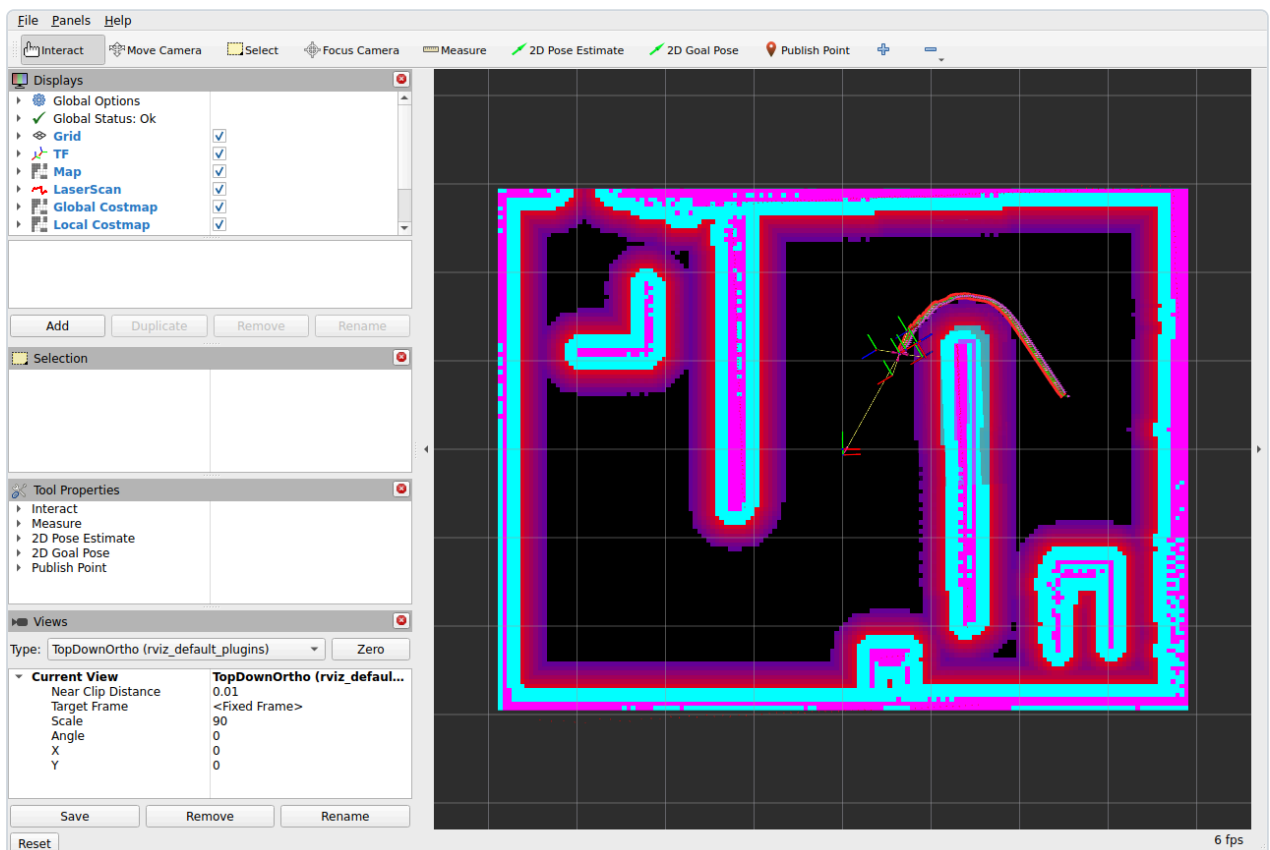
`goal_timeout_sec`（launch 里配了 240 秒，照顾 Gazebo 低 RTF）是业务层给自己上的保险。注意超时计时用的是 **ROS 时钟**（`self.get_clock().now()`）——"导航跑了多久"是仿真世界内的物理过程，仿真暂停时它就该跟着停。对照安全门用 `time.monotonic()` 的理由，正好是同一条规则的两面。

5.5 6.3 没展开的业务层

- detail 状态字符串全集**（验收脚本全靠它断言）：`ready` → `queued:x` → `sending:x` → `active:x` → 终态 `reached:x` / `failed:x` / `rejected:x` / `goal_timeout`；异常态 `waiting_for_nav2`、`unknown_goal:x`、`awaiting_confirmation`；停止态 `stopped`（用户 stop）/ `canceled`（用户 cancel）；巡航专属 `patrol_complete`、`empty_patrol`。

- **patrol 的链式推进**: `_goal_finished` 里到点后把下一个路点写回 `_pending_send` (L556~563), 重新走一遍"排队 → `_tick` 发送"的循环。巡航不是一个特殊流程, 而是"单点导航"的自我接力。
- **模式互斥**: `_enter_xxx` 每个入口都先 `_cancel_nav()` + 清 `_pending_send` + 关跟随, 防止"跟随和导航同时往 `cmd_vel` 发指令" (L339~349 注释)。
- **requires_confirmation 挂起**: 带确认要求的指令一律不执行、置 `awaiting_confirmation`, 但 `stop/cancel` 例外——**停止永远无条件执行** (L303~306)。这是给第六课语音大模型接入预留的安全钩子。
- **中文别名表** (L119~125): "厨房" → `kitchen`, "大厅"和"门口"都指向 `hall_entry`。查表前先 `normalize_goal_key` 归一化, "Living Room"、"living-room" 全部命中同一项。

在 RViz 里发一个目标后你应该看到的画面 (真实截图, 红色曲线为规划路径, 细黄线为 DWB 局部轨迹):



RViz: 发出导航目标后, 全局路径与机器人运动

走读自查: `_goal_finished` 开头有一段 `if self._mode == "idle": return` (L543~545), 删掉它会出什么 bug? (提示: 用户在导航途中发了 `stop`, 紧接着 Nav2 的 `CANCELED` 结果到达。)

第 6 章 waypoint_patrol.py——同一件事的"同步写法"

139 行, 是全课最短的节点, 放在 `mission_executor` 之后读特别有意思: 它做的事 (多点巡航) 是 `executor patrol` 模式的子集, 代码量却只有四分之一。差价买走了什么?

	waypoint_patrol.py (同步)	mission_executor.py (异步)
API	Nav2 官方 <code>BasicNavigator</code> 封装	自己写 <code>ActionClient</code>
流程	顺序执行：发目标 → 轮询等结果 → 下一个	回调驱动：随时可被新指令打断
执行中收新指令	不可能 （主线程阻塞在轮询里）	随时（这就是那四分之三的代码）
适用场景	一次性演示脚本	常驻服务

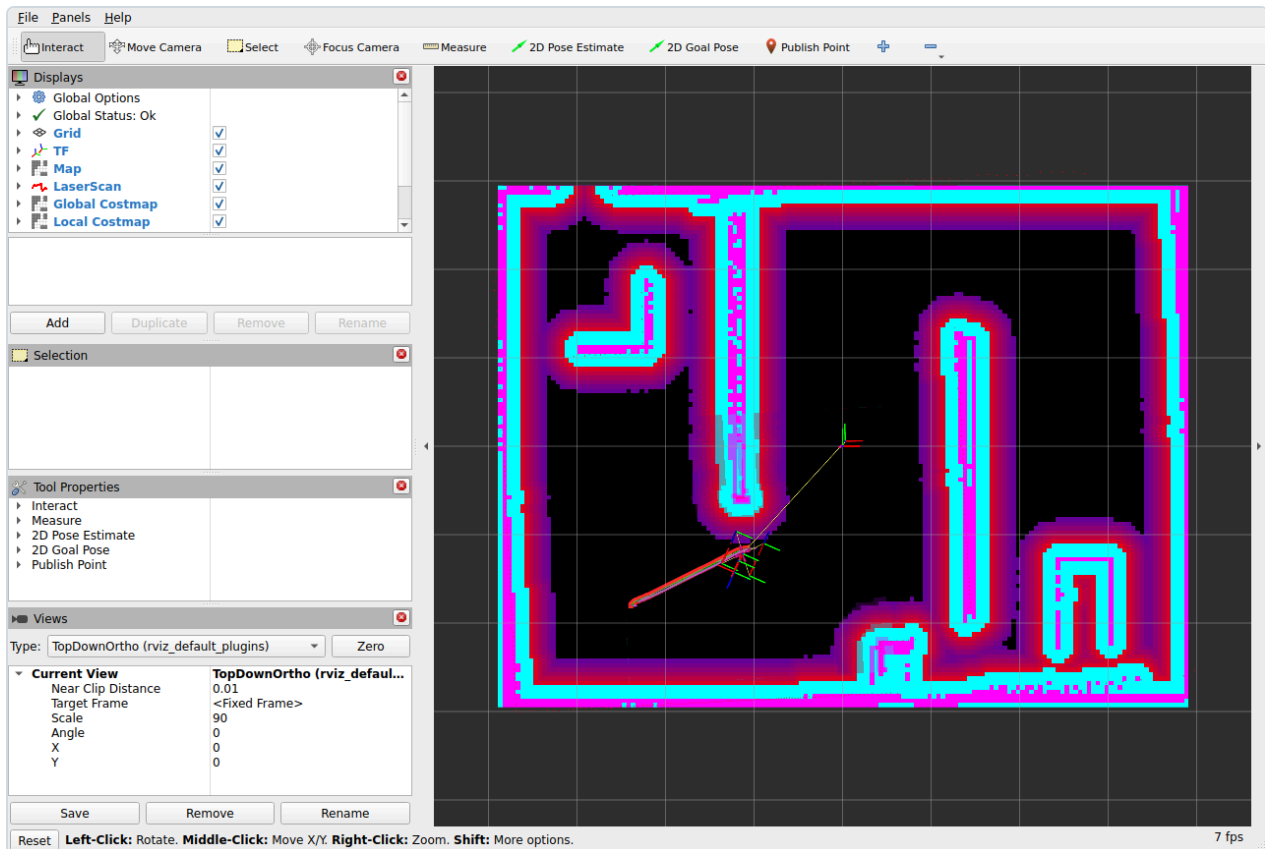
核心等待逻辑 `_wait_for_task`：

```
while not navigator.isTaskComplete():
    if timeout_sec > 0 and time.monotonic() - start > timeout_sec:
        navigator.cancelTask()          # 超时主动取消，防止卡死在一个路点
        return False
    time.sleep(0.2)                      # BasicNavigator 内部有 spin 线程，睡这里不堵消息
result = navigator.getResult()          # SUCCEEDED / CANCELED / FAILED 三种结局
```

`goToPose` / `isTaskComplete` / `getResult` 把 action 的三环回调链完全藏了起来——对照动手教程 6.1 的 v1，你会发现自己已经写过它的用法。三个工程细节：

1. **版本兼容** (L100~103)： `waitUntilNav2Active()` 在新旧 API 间用 `try/except TypeError` 切换，这是应对依赖库 API 变动的常见手法；
2. **单点失败不中断** (L125)： `_wait_for_task` 的返回值被有意忽略，某个路点到不了就跳下一个——演示场景"皮实"比"严谨"重要；
3. **善后** (L133)：结束时 `lifecycleShutdown()` 把 Nav2 全家关掉，与 `lifecycle_activator` 的激活互为逆操作。

巡航运行中的画面（真实截图，机器人正沿路径驶向下一路点）：



RViz: 多点巡航进行中

走读自查: 为什么 `time.sleep(0.2)` 在这里不会把节点"睡死", 而 `nav_acceptance` 里却必须用 `spin_once` 循环代替 `sleep` (见第 8 章)?

第 7 章 mecamind_nav2_params.yaml——参数文件导读

372 行参数不需要逐个背, 但要会读。记住一条主线: 每个顶层键对应一个 Nav2 节点, 和 launch 里的 9 个 Node 一一对应。下面按三类重点导读。

7.1 第一类: 麦轮全向底盘的三处特配

差速车的参数模板抄过来是跑不动麦轮车的, 改的就是这三处:

```

amcl:
  robot_model_type: nav2_amcl::OmnimotionModel  # 全向运动模型（差速模板是 DifferentialMotionModel）
  alpha5: 0.2                                     # 全向平移专用的噪声系数

controller_server:
  FollowPath:
    min_vel_y: -0.25      # y 方向速度非零 = 允许横移
    max_vel_y: 0.25       # 差速模板这两项是 0.0
    vy_samples: 6         # 采样也要覆盖 y 维度（差速模板是 1）

velocity_smoother:
  max_velocity: [0.35, 0.25, 1.2]  # 三分量 [x, y, theta], y 同样非零

```

课堂提问的好素材：如果只改 controller 忘了改 AMCL，会发生什么？（答：DWB 会下发横移速度，但 AMCL 的差速模型认为"机器人不可能横移"，把横移里程当成噪声——定位会在每次横移后跳变。）

7.2 第二类：WSL2 性能减配（注释里的"踩坑实录"）

```

controller_server:
  controller_frequency: 10.0  # 原 20Hz，WSL2 上会吃满 CPU 拖低 Gazebo RTF
  FollowPath:
    vx_samples: 10             # 10*6*16=960 条候选轨迹/周期
    vy_samples: 6              # 原 18*10*24=4320 条，CPU 撑不住
    vtheta_samples: 16
  amcl:
    max_beams: 80              # 360 束激光只抽 80 束打分，够用且省 CPU

```

这些数字的教学价值在于它们不是抄来的，是被"导航又慢又卡最后超时"的现场逼出来的（《从零搭建教程》12.8 节记录了完整排障过程）。参数调优的第一课：先知道每个参数烧多少 CPU。

7.3 第三类：容易看漏的设计取舍

参数	值	背后的决定
local_costmap.global_frame	odom（不是 map！）	控制回路要连续坐标；AMCL 修正会让 map 系位姿跳变，局部避障受不了
amcl.update_min_d / update_min_a	0.15	移动 0.15m 或转 0.15rad 才更新一次滤波——这就是 map→odom 修正呈阶梯状跳变的原因
amcl.set_initial_pose	false	初始位姿不写死在参数里，由 lifecycle_activator 运行时发布（单一事实来源）
inflation_radius	0.45	特意调大：让 DWB 轨迹离墙远一点，避免安全门 warn 区反复触发"减速—恢复"拉锯
general_goal_checker.xy_goal_tolerance	0.20	进 0.2m 算到达——named_goals 坐标不必精确到厘米的底气来源
lifecycle_manager_navigation	（整段）	备而不用：官方管理器的配置留着，本项目实际用自研激活器（对照学习用）

`inflation_radius: 0.45` 那一条尤其值得展开：它是**跨节点调参**的例子——调的是 Nav2 代价地图，解决的却是自研安全门的问题（warn 线 0.45m 和它数值相同不是巧合）。两道防线的参数必须互相咬合：代价地图负责“平时就不靠近”，安全门负责“万一靠近了兜底”。

走读自查：`global_costmap` 有 `static_layer` 而 `local_costmap` 没有，`local_costmap` 是 `rolling_window` 而 `global` 不是——把这两个差异用一句话解释清楚。

第 8 章 验收链：nav_acceptance.py + test_cp3.sh——系统怎么给自己出考卷

最后走读“自动考官”。它的教学价值不在导航，而在**测试思想**：怎么无人值守地证明一套异步系统真的能用？

8.1 黑盒原则：只走学生走的路

nav_acceptance 与系统的全部接口：

发布	/mecamind/task_command	(和你手敲 <code>ros2 topic pub</code> 完全一样)
订阅	/mecamind/mission_state	(断言依据)
订阅	/amcl_pose	(证据：定位是否漂了)
订阅	/mecamind/safety_state	(证据：是不是被安全门拦停的)

它不直接调 Nav2 action——如果直接调，测试就绕过了 mission_executor，"executor 有 bug 但验收通过"这种事就可能发生。黑盒测试覆盖的是学生实际使用的完整链路。

8.2 同步测试脚本的两个基本功

基本功一：边等边泵回调。 测试主流程是顺序的（发指令 → 等结果 → 下一步），但等待期间订阅回调必须照常执行，否则状态缓存不更新、永远等不到结果：

```
def _spin_for(self, seconds: float) -> None:
    end = time.monotonic() + seconds
    while time.monotonic() < end and rclpy.ok():
        rclpy.spin_once(self, timeout_sec=0.1)  # 不能用裸 time.sleep!
```

（对照第 6 章自查题：waypoint_patrol 敢睡是因为 BasicNavigator 内部有独立 spin 线程；这里的节点没有，睡了就聋了。）

基本功二：把“等什么”参数化。 等受理、等到点、等停止，逻辑全是“轮询直到状态满足条件”，差别只在条件本身——所以把条件做成传入的谓词函数：

```
self.wait_for_detail(
    lambda s: s.get("detail") == f"reached:{name}"
    or (s.get("mode") == "idle" and str(s.get("detail", "")).startswith(("failed:", "rejected:", ...))),
    timeout, f"goal {name}",
)
```

注意这个谓词等的是**“有结果”**（成功或失败都算），拿到结果后再判断是不是成功——如果只等 `reached:`，失败场景会干等到超时，浪费 3 分钟才报错。

还有一个容易漏的细节（L185~193）：每次发指令后先等 `queued:/sending:/active:` 之一出现，**确认本次任务已被受理**，再等终态——否则上一次尝试残留的 `failed:xxx` 会被误读成本次结果。异步系统的断言必须先“对齐时序”再“读结果”。

8.3 失败时留证据

```
if not report.get("overall_ok"):
    report["evidence"] = {
        "last_amcl_pose": self._last_amcl,      # 定位漂了吗？
        "last_safety_state": self._last_safety, # 被安全门拦了吗？
        "mission_state_tail": self._state_history[-15:], # 失败前发生了什么？
    }
```

验收失败最怕“复现不了”。报告里塞进最后的定位、安全门状态和最近 15 条任务状态，课后不用重跑就能离线还原故障现场。`_state_cb` 里的**去重**（只记录变化了的状态，L86）也是配套设计——`mission_executor` 每 0.5 秒心跳重发，不去重的话历史里全是重复条目。

8.4 test_cp3.sh：把一切串成一条命令

```
./scripts/test_cp3.sh # lite 后端，约 3 分钟
MECAMIND_CP3_BACKEND=gazebo ./scripts/test_cp3.sh # Gazebo headless，约 10 分钟
```

脚本流程：**锁** → **起系统** → **跑考官** → **解析报告** → **清理**。四个 shell 工程点：

1. **锁目录防并发**（L16）：`mkdir` 是原子操作，天然适合当锁；锁里存 PID，持锁进程已死时自动清锁——防止上次异常退出把锁留成“死锁”；
2. **无头启动**（L77~81）：`use_gui:=false with_rviz:=false`，后台运行、输出重定向到日志文件；
3. **按后端区分超时**（L91~94）：lite 单目标 180 秒、Gazebo 420 秒——WSL2 上 Gazebo RTF 常低于 1，墙钟超时必须放宽；
4. **trap cleanup EXIT**（L70）：无论脚本怎么退出（成功、失败、Ctrl+C），都会递归找出 launch 的全部子进程逐个 INT→TERM→KILL——ROS launch 的进程树不清干净，下次运行就会撞上僵尸节点。

通过时的输出长这样：

[CP3] [正常] 目标 living_room: reached:living_room 用时 41.3s
[CP3] [正常] 目标 bedroom: reached:bedroom 用时 58.7s
[CP3] [正常] 目标 hall_entry: reached:hall_entry 用时 47.2s
[CP3] 取消测试: activated=True stopped=True
MECAMIND_CP3_OK

报告文件在 `maps/mecamind_cp3_lite_nav_report.json`，`overall_ok: true` 即 PASS。

走读自查：验收流程里"取消测试"为什么要先 `_spin_for(3.0)` 让机器人真跑起来 3 秒再发 `stop`，而不是发完目标立刻取消？（提示：想想 `queued:` 状态下取消和 `active:` 状态下取消，考的是不是同一段代码。）

附录 A：全部状态字符串速查

验收、排障都靠读 `/mecamind/mission_state` 的 `detail` 字段，全集如下：

detail	含义	出处
ready	刚启动	<code>__init__</code>
waiting_for_nav2	目标已排队，等 Nav2 action 服务端上线	<code>_tick</code>
queued:x / sending:x / active:x	排队 / 发送中 / 导航中	<code>_enter_*</code> / <code>_send_pending</code> / <code>_goal_response</code>
reached:x	单点到达（终态）	<code>_goal_finished</code>
failed:x / rejected:x / send_failed:x / result_error:x	各类失败（终态）	<code>_goal_finished</code> / <code>_goal_response</code>
goal_timeout	超时被业务层取消	<code>_poll_active_goal</code>
unknown_goal:x	目标名查不到表	<code>_enter_navigate</code>
awaiting_confirmation	指令带确认要求，挂起	<code>_task_cb</code>
stopped / canceled	用户 stop / cancel	<code>_stop_all</code>
patrol_complete / empty_patrol	巡航走完 / 路点表为空	<code>_goal_finished</code> / <code>_enter_patrol</code>
follow_enabled	跟随模式开启	<code>_enter_follow</code>

附录 B：课堂串讲建议（90 分钟版）

时间	内容	素材
0~10 min	rqt_graph 活架构图，三条线走一遍	0.1 节截图
10~25 min	launch 走读：remapping 三处 + TimerAction + 旁路思想实验	第 1 章
25~40 min	安全门：四扇区、分量原则、两个看门狗、monotonic vs ROS 时钟	第 4 章
40~60 min	任务执行器：三个补丁逐个对照 v2 讲	第 5 章 + 动手教程 6.3
60~70 min	waypoint_patrol 对照：同步 vs 异步的代价	第 6 章
70~80 min	参数导读：麦轮三处 + inflation 与安全门的咬合	第 7 章
80~90 min	现场跑 test_cp3.sh，等结果期间讲验收链设计	第 8 章

每章末尾的"走读自查"共 8 题，适合当课后作业或下节课开场提问。